BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CMC
KHOA CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO HỌC PHẦN & SỔ TAY KỸ THUẬT DEVOPS
HỌC PHẦN: TRIỂN KHAI PHẦN MỀM (SOFTWARE DEPLOYMENT)

ĐỀ TÀI:
THIẾT KẾ VÀ THỰC THI CHUỖI TỰ ĐỘNG HÓA CI/CD, ĐÓNG GÓI DOCKER, QUẢN TRỊ VẬN HÀNH & GIÁM SÁT HỆ THỐNG WORKSPHERE

STT	Họ và Tên Thành Viên	Mã Sinh Viên	Vai trò môn học
1	Nguyễn Minh Hiếu	BIT247729	Nhóm trưởng chính thức
2	Nguyễn Văn Cường	BIT240048	Thành viên
3	Vũ Thị Thu Hướng	BIT240113	Thành viên
4	Hoàng Bảo Long	BIT240142	Thành viên
5	Nguyễn Đức Khiêm	BIT240125	Thành viên

(Trạng thái phân công: PROPOSED_ASSIGNMENT_PENDING_USER_APPROVAL — Phân công biên soạn và thuyết trình đề xuất chờ Người dùng phê duyệt)

HÀ NỘI — NĂM 2026

MỤC LỤC & MA TRẬN ÁNH XẠ 9 BÀI GIẢNG HỌC PHẦN

Học phần Triển khai phần mềm (Khoa CNTT & TT — Trường Đại học CMC) bao gồm 9 chuyên đề bài giảng. Bảng dưới đây đối soát toàn diện lý thuyết từng bài với hiện trạng kỹ thuật thực tế của dự án WorkSphere:

Bài học / Chuyên đề	Khái niệm cốt lõi	Hiện trạng WorkSphere	Minh chứng kỹ thuật	Phân loại nhãn
Bài 1: Giới thiệu DevOps	Văn hóa phối hợp, tự động hóa, phản hồi nhanh	Áp dụng vòng lặp DevOps, wrappers, CI/CD, runbooks	compose.yaml, scripts/	IMPLEMENTED_PARTIALLY
Bài 2: Cloud Computing	Mô hình IaaS, PaaS, SaaS, Public/Hybrid Cloud	Phân tích kịch bản Staging/Production đám mây tương lai	Chương 10, DECISIONS.md	CONCEPT_ONLY
Bài 3: SCM & Git	Kiểm soát phiên bản, commit, branch, merge, tag	Git repository chuẩn hóa, commit SHA cố định, .gitignore	Git history, .gitignore	IMPLEMENTED
Bài 4: Continuous Integration	Tự động hóa build, test, secret scan, gate	GitHub Actions 5 jobs song song (351 backend + 1144 jest)	.github/workflows/ci.yml	IMPLEMENTED
Bài 5: Continuous Delivery	Đóng gói phát hành, phân định CD vs Deployment	Đóng gói Docker image, APK debug; chưa auto-deploy	ci.yml (Artifacts)	PARTIAL
Bài 6: Infrastructure as Code	Khai báo hạ tầng dạng mã, tính tái lặp	Dockerfile, compose.yaml, prometheus.yml, scripts	compose.yaml, Dockerfile	PARTIAL
Bài 7: Docker Container	Image, Container, Multi-stage, Non-root, Health	Multi-stage JRE 21, non-root UID 10001, curl healthcheck	backend/Dockerfile	IMPLEMENTED
Bài 8: Kubernetes	Pod, Deployment, Service, PVC, Ingress, Scaling	Mô hình hóa lý thuyết ánh xạ từ Compose sang K8s	Chương 10, Sơ đồ Fig 8	CONCEPT_ONLY
Bài 9: Monitoring & Logging	Metrics, Log correlation, Health, Prometheus	Actuator /prometheus, X-Request-ID MDC, 15s scrape	prometheus.yml, Actuator	IMPLEMENTED_LOCAL_INTERNAL

Báo cáo này được cấu trúc theo mô hình Sổ tay kỹ thuật (Technical Handbook), phục vụ công tác bàn giao và bảo vệ đồ án của nhóm 5 sinh viên.

CHƯƠNG 1: TỔNG QUAN VĂN HÓA DEVOPS & DỰ ÁN WORKSPHERE

1.1. Bản Chất Của Văn Hóa DevOps & Chu Trình Cải Tiến Liên Tục

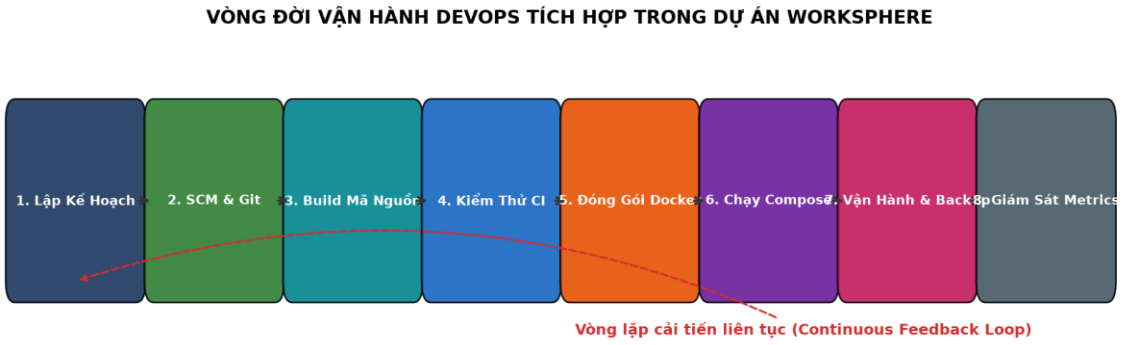
DevOps không chỉ đơn thuần là việc sử dụng công cụ tự động hóa, mà trước hết là một sự chuyển dịch văn hóa tổ chức kỹ thuật phần mềm:

- • Phá vỡ ranh giới truyền thống (Silos): Kết nối chặt chẽ giữa đội ngũ Phát triển (Development) và đội ngũ Vận hành (Operations), xóa bỏ tình trạng 'ném mã nguồn qua bức tường' để bên kia tự triển khai.
- • Vòng lặp phản hồi nhanh (Fast Feedback Loops): Phát hiện lỗi lập trình, lỗi bảo mật và sự cố tích hợp ngay từ giai đoạn sớm (Shift-Left Quality) thay vì chờ đến khi đưa lên máy chủ thử nghiệm.
- • Tự động hóa toàn diện (Automation): Chuẩn hóa môi trường thông qua mã lệnh, loại bỏ các bước thao tác thủ công dễ gây sai sót con người (Human Error).

1.2. Áp Dụng DevOps Trong Hệ Thống Quản Trị Doanh Nghiệp WorkSphere

Dự án WorkSphere áp dụng triệt để các trụ cột DevOps hiện đại:

- • Tính xác định (Determinism): Sử dụng Maven Wrapper (`mvnw`), Gradle Wrapper (`gradlew`) và tệp khóa `.node-version` (Node 24.19.0) nhằm đảm bảo mã nguồn biên dịch ra cùng một kết quả trên mọi máy trạm phát triển.
- • Kiểm soát chất lượng liên tục: Toàn bộ mã nguồn phải vượt qua 0 lỗi Checkstyle, 351 Backend Tests và 1,144 Mobile Jest Tests trước khi được phép tích hợp.
- • Đóng gói độc lập nền tảng: Sử dụng Multi-stage Dockerfile tạo ra container image khép kín, sẵn sàng chạy ngay trên môi trường Docker Compose mà không đòi hỏi cài đặt thủ công Java hay PostgreSQL trên máy chủ.



Hình 1.1: Chu trình DevOps khép kín áp dụng trong dự án WorkSphere

CHƯƠNG 2: QUẢN LÝ MÃ NGUỒN (SCM) & CẤU HÌNH HỆ THỐNG

2.1. Quản Trị Phiên Bản Với Git & Nguyên Tắc Bất Biến (Immutable SHA)

Hệ sinh thái WorkSphere đặt toàn bộ mã nguồn, tệp cấu hình và tài liệu kỹ thuật dưới sự kiểm soát chặt chẽ của hệ thống Git:

- • Kho lưu trữ chính: Nhánh `main` là nhánh phát triển và tích hợp tập trung của toàn dự án.
- • Định danh bất biến (Commit SHA): Mọi mốc nghiệm thu kỹ thuật (Checkpoints) đều được ghim chính xác bằng mã băm 40 ký tự SHA (ví dụ: `37c4d7e...`), ngăn chặn hoàn toàn việc sửa đổi lịch sử.

- • Chính sách bảo vệ kho mã (.gitignore): Loại bỏ 100% các tệp nhạy cảm (`.env`, private keys), tệp nhị phân build thừa (`target/`, `node_modules/`, `build/`), và đặc biệt là 4 thư mục học liệu gốc được giữ nguyên trạng thái `LOCAL_ONLY_REFERENCE`.

2.2. Chiến Lược Phân Nhánh & Quản Lý Cấu Hình Môi Trường (.env)

Nguyên tắc quản trị cấu hình Twelve-Factor App:

- • Tách biệt mã nguồn và cấu hình: Mã nguồn tuyệt đối không chứa mật khẩu hay thông tin định danh máy chủ. Toàn bộ tham số kết nối được nạp qua biến môi trường runtime.
- • Mẫu cấu hình chuẩn `.env.example`: Cung cấp danh mục các biến môi trường cần thiết kèm giá trị mẫu an toàn, giúp thành viên mới nhanh chóng thiết lập môi trường phát triển chỉ trong vài phút.

```
# Trích đoạn .env.example chuẩn hóa trong WorkSphere
POSTGRES_DB=worksphere
POSTGRES_USER=postgres
POSTGRES_PASSWORD=change_this_in_production
JWT_SECRET=super_secret_jwt_key_at_least_32_characters_long
API_PORT=8080
```

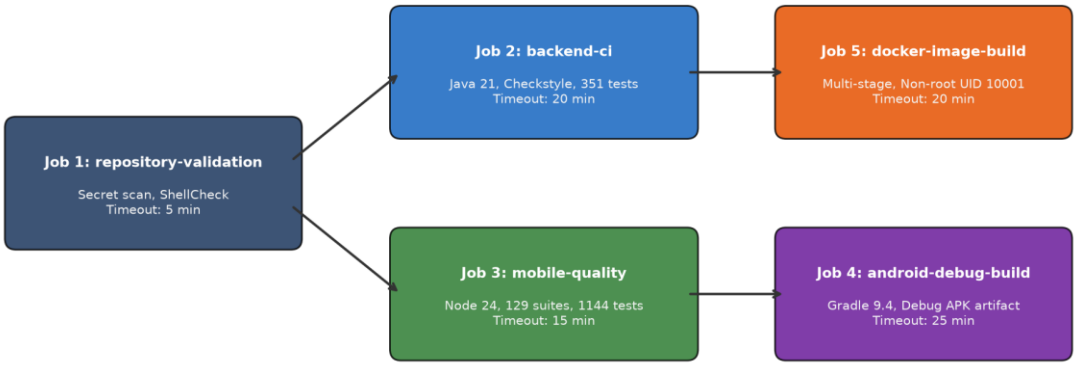
CHƯƠNG 3: TỰ ĐỘNG HÓA TÍCH HỢP LIÊN TỤC (CI) VỚI GITHUB ACTIONS

3.1. Kiến Trúc Pipeline CI 5 Jobs Độc Lập

Pipeline CI của WorkSphere được định nghĩa khai báo trong tệp `.github/workflows/ci.yml`. Để tối ưu hóa thời gian thực thi, pipeline được phân rã thành 5 jobs chuyên biệt có đồ thị phụ thuộc (DAG) chặt chẽ:

Job CI Name	Tiền nhiệm	Môi trường & Công cụ	Nhiệm vụ thực thi	Thời gian giới hạn
repository-validation	Không	Ubuntu Latest, ShellCheck, Secret Scan	Quét rò rỉ mật khẩu, kiểm tra cấu trúc thư mục, audit shell	Timeout: 5 phút
backend-ci	Job 1	Ubuntu Latest, Java 21 Temurin, Maven	Kiểm tra Checkstyle 0 lỗi, chạy 351 unit & slice tests	Timeout: 20 phút
mobile-quality	Job 1	Ubuntu Latest, Node.js 24.19.0, npm	Chạy ESLint, TypeScript check, 129 Jest suites / 1,144 tests	Timeout: 15 phút
android-debug-build	Job 1, 3	Ubuntu Latest, Android SDK 35/37, Gradle	Biên dịch Gradle assembleDebug, tạo app-debug.apk artifact	Timeout: 25 phút
docker-image-build	Job 1, 2	Ubuntu Latest, Docker Buildx	Build Multi-stage Docker image, xác thực non-root UID 10001	Timeout: 20 phút

CẤU TRÚC 5 JOBS TRONG PIPELINE GITHUB ACTIONS CI CỦA WORKSPHERE



Hình 3.1: Đồ thị thực thi 5 jobs tự động hóa trong GitHub Actions CI

3.2. Chốt Chặn Kiểm Tra Bí Mật & Chất Lượng Mã Nguồn (Quality Gates)

Mỗi job trong CI đóng vai trò như một chốt chặn chất lượng không thể vượt qua nếu có lỗi phát sinh:

- • Fail-Fast: Bất kỳ vi phạm nào về Checkstyle, rò rỉ biến môi trường hoặc chỉ 1 ca kiểm thử bị hỏng đều khiến toàn bộ pipeline dừng lại ngay lập tức, ngăn ngừa mã nguồn lỗi hòa trộn vào nhánh `main`.
- • Lưu trữ Artifact an toàn: Bản build `app-debug.apk` được tự động đóng gói và lưu trữ trên GitHub Actions trong 7 ngày phục vụ kiểm thử nội bộ, phân định rõ ràng không phải là bản phát hành thương mại có chữ ký số.

CHƯƠNG 4: PHÂN ĐỊNH CONTINUOUS DELIVERY VÀ CONTINUOUS DEPLOYMENT

4.1. Phân Biệt Giữa Tự Động Hóa Giao Hàng (CD) Và Triển Khai (Deployment)

Trong kỹ thuật DevOps, sự phân định giữa Continuous Delivery và Continuous Deployment là ranh giới mang tính sống còn đối với sự an toàn của hệ thống doanh nghiệp:

- • Continuous Delivery (Giao hàng liên tục): Đảm bảo mã nguồn sau khi vượt qua CI luôn luôn nằm ở trạng thái sẵn sàng phát hành (Releasable State). Quá trình đưa lên môi trường thực tế cần một quyết định phê duyệt có kiểm soát của con người (Manual Approval Gate).
- • Continuous Deployment (Triển khai liên tục): Tự động hóa 100% việc đưa mọi commit được chấp thuận thẳng lên môi trường Production mà không cần bất kỳ sự can thiệp thủ công nào.

4.2. Định Vị Hiện Trạng Của Dự Án WorkSphere

[CLAIM_BOUNDARY] Đối soát trung thực với năng lực kho mã hiện hành [CLAIM BOUNDARY]:

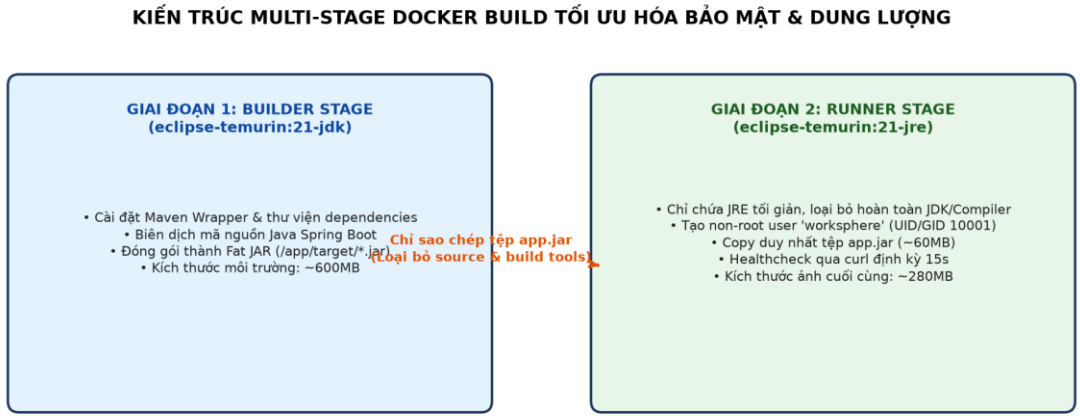
- • CI: ĐÃ HOÀN THÀNH (IMPLEMENTED) — 5 jobs GitHub Actions hoạt động ổn định.
- • Continuous Delivery Packaging: MỘT PHẦN (PARTIAL) — Đã tự động tạo Docker image cục bộ và APK debug artifact, sẵn sàng đưa vào quy trình phát hành.
- • Automatic Production Deployment: CHƯA TRIỂN KHAI (NOT_IMPLEMENTED) — WorkSphere chưa cấu hình tự động đẩy image lên registry công khai hay triển khai tự động lên cụm máy chủ đám mây. Quyết định này phù hợp với phạm vi an toàn của đồ án học thuật.

CHƯƠNG 5: ĐÓNG GÓI DOCKER IMAGE & KỸ THUẬT MULTI-STAGE BUILD

5.1. Tối Ưu Hóa Dung Lượng & Bảo Mật Với Multi-Stage Build

Tệp `backend/Dockerfile` áp dụng kỹ thuật Multi-stage Build tối tân nhằm giải quyết đồng thời hai bài toán: giảm kích thước image và gia cố bảo mật:

- • Giai đoạn 1 - Builder (`eclipse-temurin:21-jdk`): Chứa đầy đủ bộ công cụ JDK và Maven để tải dependencies và biên dịch mã nguồn. Giai đoạn này tiêu tốn hơn 600MB dung lượng.
- • Giai đoạn 2 - Runner (`eclipse-temurin:21-jre`): Chỉ sử dụng bộ thực thi JRE tối giản, loại bỏ hoàn toàn mã nguồn gốc, tệp tạm và trình biên dịch. Kích thước image cuối cùng giảm xuống chỉ còn ~280MB.



Hình 5.1: Kiến trúc đóng gói hai giai đoạn (Multi-stage) của Dockerfile

5.2. An Toàn Thực Thi Với Tài Khoản Phi Đặc Quyền (Non-Root User)

Một trong những lỗ hổng phổ biến nhất trong container hóa là chạy ứng dụng dưới quyền `root`. WorkSphere thiết lập cơ chế phòng vệ chuyên sâu:

- • Tạo nhóm và tài khoản chuyên biệt: Tạo user `worksphere` với `UID:GID = 10001:10001`.
- • Chuyển quyền thực thi: Chỉ thị `USER worksphere` bảo đảm nếu ứng dụng Spring Boot bị tấn công khai thác mã từ xa (RCE), kẻ tấn công cũng không thể chiếm quyền kiểm soát máy chủ Docker host.
- • Tích hợp cơ chế tự kiểm tra sức khỏe (Container Healthcheck): Sử dụng lệnh `curl` thăm dò endpoint `/actuator/health` định kỳ 15 giây một lần để Docker Engine tự động nhận biết tình trạng sống sót của container.

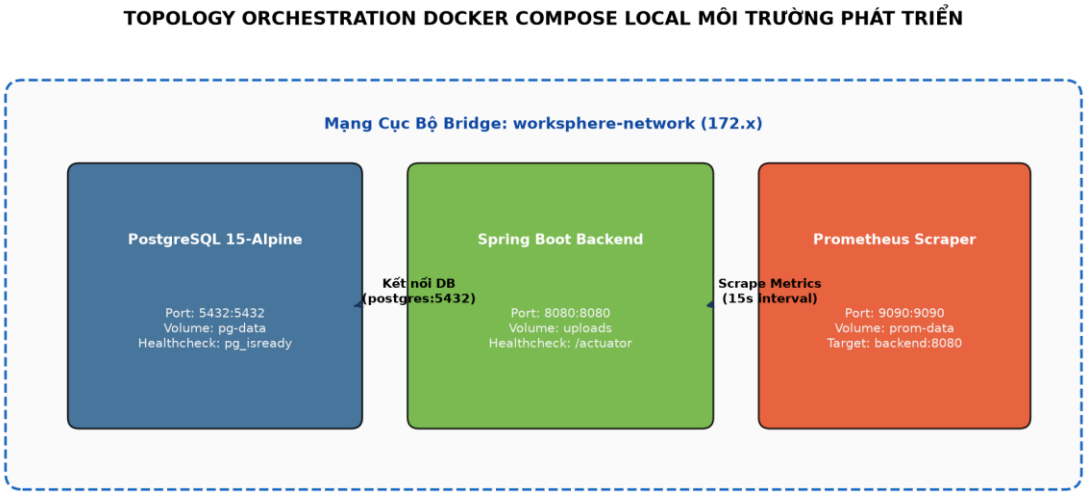
```
# Trích đoạn khai báo bảo mật trong backend/Dockerfile
RUN groupadd -g 10001 worksphere && \
    useradd -u 10001 -g worksphere -m -s /bin/bash worksphere
USER worksphere
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=5s --start-period=30s --retries=3 \
  CMD curl -f http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

CHƯƠNG 6: ORCHESTRATION CỤC BỘ VỚI DOCKER COMPOSE

6.1. Cấu Trúc Khai Báo Dịch Vụ Trong compose.yaml

Tệp `compose.yaml` cung cấp môi trường chạy độc lập, khép kín cho toàn bộ hệ sinh thái cục bộ:

- • Dịch vụ `postgres`: Chạy PostgreSQL 15-Alpine, ánh xạ cổng `5432:5432` cho máy host và gắn volume bền vững `worksphere-postgres-data`.
- • Dịch vụ `backend`: Chạy image Spring Boot, ánh xạ cổng `8080:8080`, phụ thuộc vào tình trạng khỏe mạnh của PostgreSQL (`depends_on.postgres.condition: service_healthy`).
- • Mạng cầu nối (Bridge Network): Tạo mạng riêng `worksphere-network`, cho phép các container phân giải tên miền nội bộ qua Docker DNS (ví dụ: `postgres:5432`).



Hình 6.1: Sơ đồ mạng và volume bền vững của Docker Compose

6.2. Phân Định Khởi Chạy Và Dọn Dẹp Sạch Sẽ (Persistence vs Clean Teardown)

Quy chuẩn vận hành dữ liệu an toàn:

- • Dừng thông thường (`docker compose down`): Dừng và gỡ bỏ các container nhưng bảo lưu nguyên vẹn dữ liệu trong named volumes.
- • Dọn dẹp triệt để (`docker compose down -v`): Xóa sạch cả các volume dữ liệu, chỉ sử dụng khi cần khởi tạo lại môi trường từ đầu hoặc phục vụ kịch bản kiểm thử cách ly.

CHƯƠNG 7: CƠ SỞ HẠ TẦNG DƯỚI DẠNG MÃ (IAC): ĐÁNH GIÁ MỨC ĐỘ TRIỂN KHAI

7.1. Khái Niệm Infrastructure as Code (IaC) & Lợi Ích Cốt Lõi

Infrastructure as Code là phương pháp quản trị hạ tầng điện toán thông qua các tệp mã nguồn khai báo có thể theo dõi phiên bản, thay vì cấu hình thủ công bằng giao diện đồ họa. IaC mang lại tính nhất quán, khả năng tái lập và giảm thiểu rủi ro lệch cấu hình (Configuration Drift).

7.2. Định Vị Mức Độ Triển Khai IaC Của Dự Án WorkSphere

[CLAIM_BOUNDARY] Bảng đánh giá mức độ hoàn thiện năng lực IaC của dự án [CLAIM BOUNDARY]:

Thành phần hạ tầng	Công cụ / Tệp cấu hình	Mức độ tự động hóa & Phạm vi áp dụng	Phân loại năng lực
Đóng gói môi trường Runtime	backend/Dockerfile	Khai báo JRE, cổng, quyền user, healthcheck bằng mã	IMPLEMENTED
Orchestration đa dịch vụ	compose.yaml	Khai báo tự động DB, Backend, Volumes, Network	IMPLEMENTED_LOCAL_ONLY
Tự động hóa chuỗi CI/CD	.github/workflows/ci.yml	Khai báo 5 jobs, dependency DAG, timeout, caching	IMPLEMENTED
Giám sát & Scrape Metrics	prometheus.yml	Khai báo target, tần suất scrape 15s bằng file YAML	IMPLEMENTED_LOCAL_INTERNAL
Cung cấp hạ tầng Đám mây	Terraform / Ansible / Pulumi	Chưa áp dụng công cụ cấp phát tài nguyên cloud tự động	NOT_IMPLEMENTED

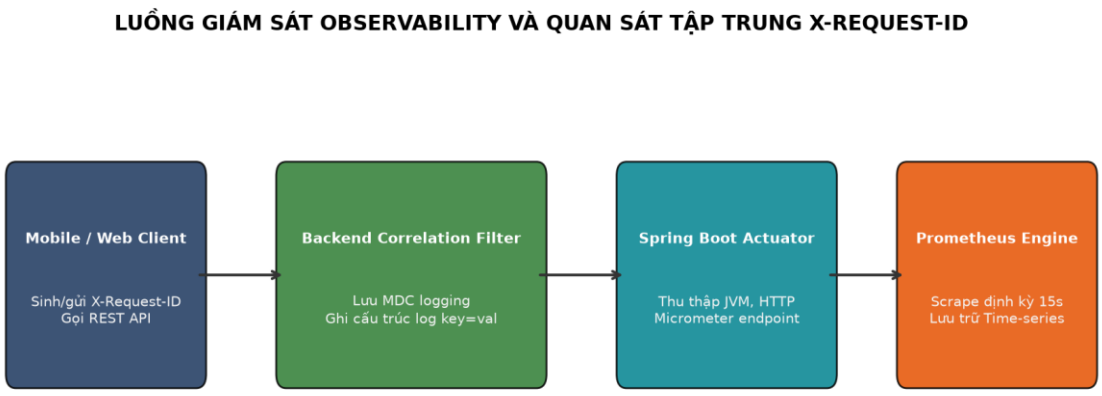
Kết luận: Mức độ trưởng thành IaC của dự án đạt cấp độ PARTIAL (Hoàn thiện cấu hình khai báo ở tầng container và pipeline CI/CD cục bộ; bảo lưu công cụ cấp phát tài nguyên cloud Terraform/Ansible cho giai đoạn phát triển tiếp theo).

CHƯƠNG 8: GIÁM SÁT HỆ THỐNG (MONITORING), LOGGING VÀ PROMETHEUS

8.1. Thu Thập Số Liệu Với Spring Boot Actuator & Prometheus

WorkSphere triển khai hạ tầng giám sát thời gian thực dựa trên chuẩn công nghiệp:

- • Spring Boot Actuator: Tích hợp thư viện Micrometer, tự động thu thập các chỉ số nội tại của ứng dụng gồm: tỷ lệ chiếm dụng bộ nhớ JVM (Heap/Non-heap), tải CPU, số lượng kết nối Connection Pool (HikariCP) và thời gian phản hồi HTTP request.
- • Prometheus Engine: Cấu hình scrape định kỳ 10-15 giây một lần tới endpoint `/actuator/prometheus`, lưu trữ dữ liệu dạng Time-series phục vụ phân tích xu hướng tải.



Hình 8.1: Quy trình thu thập số liệu giám sát và định danh yêu cầu xuyên suốt

8.2. Định Danh Yêu Cầu Xuyên Suốt (Request Correlation với X-Request-ID)

Để giải quyết bài toán truy vết lỗi trong kiến trúc phân tán giữa Client di động và Backend máy chủ, dự án áp dụng bộ lọc `RequestCorrelationFilter`:

- • Cơ chế hoạt động: Khi request đi vào, filter kiểm tra header `X-Request-ID`. Nếu thiếu, hệ thống tự động sinh một mã UUID duy nhất.
- • Lưu trữ ngữ cảnh MDC (Mapped Diagnostic Context): Mã định danh được gắn vào luồng xử lý (ThreadLocal), bảo đảm mọi dòng log sinh ra trong quá trình xử lý nghiệp vụ đều chứa trường `requestId=...`.
- • Phản hồi Client: Header `X-Request-ID` được trả lại trong HTTP response, cho phép người dùng hoặc kỹ sư đối soát chính xác nguyên nhân lỗi khi xảy ra sự cố.

8.3. Ranh Giới An Toàn Giám Sát [SECURITY BOUNDARY]

[SECURITY_BOUNDARY] Tuân thủ nguyên tắc tối thiểu hóa bề mặt tấn công:

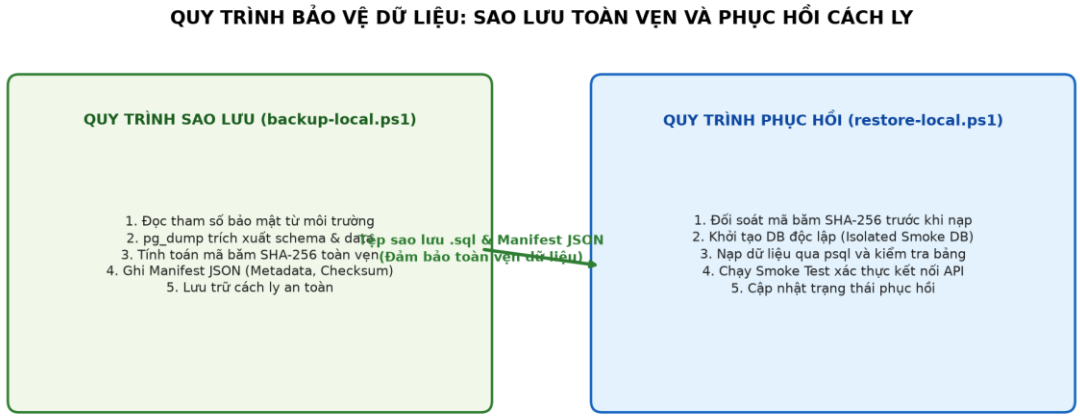
- • Chỉ công khai endpoint `/actuator/health` và `/actuator/info` cho các dịch vụ bên ngoài kiểm tra liveness.
- • Endpoint `/actuator/prometheus` chỉ mở trong mạng nội bộ Docker (`127.0.0.1` hoặc nội bộ bridge), không bind ra ngoài Internet.
- • Chưa triển khai giao diện Grafana và quy tắc cảnh báo (Alerting Rules) — được phân loại hoãn lại cho môi trường Staging/Production.

CHƯƠNG 9: QUẢN TRỊ DỮ LIỆU: SAO LƯU, PHỤC HỒI VÀ QUY TRÌNH ROLLBACK

9.1. Quy Trình Sao Lưu Toàn Vẹn (Automated Backup & Checksum)

Kịch bản PowerShell `scripts/backup-local.ps1` thực thi chu trình sao lưu dữ liệu tự động:

- • Trích xuất CSDL: Sử dụng công cụ `pg_dump` tạo bản snapshot đầy đủ gồm cấu trúc bảng (schema) và toàn bộ dữ liệu.
- • Kiểm tra toàn vẹn (Integrity Check): Tự động tính toán mã băm SHA-256 cho tệp nén `.sql` ngay sau khi tạo.
- • Tệp Manifest: Lưu trữ siêu dữ liệu gồm ngày giờ, phiên bản mã nguồn Git SHA, kích thước và mã băm SHA-256 vào tệp `backup-manifest.json`.



Hình 9.1: Quy trình bảo vệ dữ liệu hai chiều: Sao lưu toàn vẹn và Phục hồi cách ly

9.2. Diễn Tập Phục Hồi Dữ Liệu Cách Ly (Isolated Restore Rehearsal)

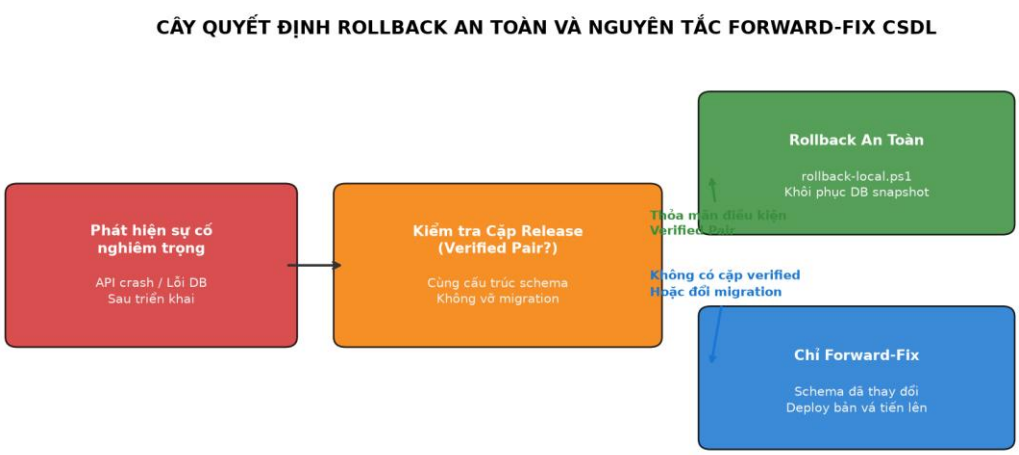
Nguyên tắc cốt lõi của DevOps: Một bản sao lưu chưa từng được diễn tập phục hồi thành công thì không được coi là một bản sao lưu an toàn:

- • Kịch bản `scripts/restore-local.ps1`: Tự động đối soát mã băm SHA-256 của tệp sao lưu với Manifest trước khi nạp dữ liệu.
- • Khởi tạo môi trường cách ly (Smoke Test DB): Tạo cơ sở dữ liệu tạm thời để nạp thử, chạy kịch bản kiểm tra số lượng bản ghi tại các bảng trọng yếu (Users, Projects, Tasks). Chỉ khi dữ liệu nguyên vẹn mới xác nhận thành công.

9.3. Cây Quyết Định Rollback & Giới Hạn Của Flyway Migration

Quy trình hoàn nguyên hệ thống khi phát sinh sự cố sau triển khai (`scripts/rollback-local.ps1`):

- • Điều kiện Rollback an toàn: Chỉ được phép rollback tự động khi tồn tại một Cặp Phát Hành Đã Được Xác Minh (Verified Release Pair) — tức phiên bản phần mềm cũ và mới có cùng cấu trúc lược đồ CSDL.
- • Nguyên tắc Forward-Only Migration của Flyway: Nếu phiên bản mới đã thực thi các script Flyway migration làm thay đổi bảng dữ liệu, việc rollback mã nguồn cũ sẽ gây lỗi xung đột không tương thích. Trong trường hợp này, đội ngũ bắt buộc phải áp dụng chiến lược 'Forward-Fix' (phát triển bản vá mới và triển khai tiến lên) thay vì rollback ngược.



Hình 9.2: Cây quyết định Rollback an toàn và nguyên tắc Forward-Fix CSDL

CHƯƠNG 10: PHÂN TÍCH ĐIỆN TOÁN ĐÁM MÂY (CLOUD) VÀ KUBERNETES

10.1. Khảo Sát Mô Hình Điện Toán Đám Mây (IaaS / PaaS / SaaS)

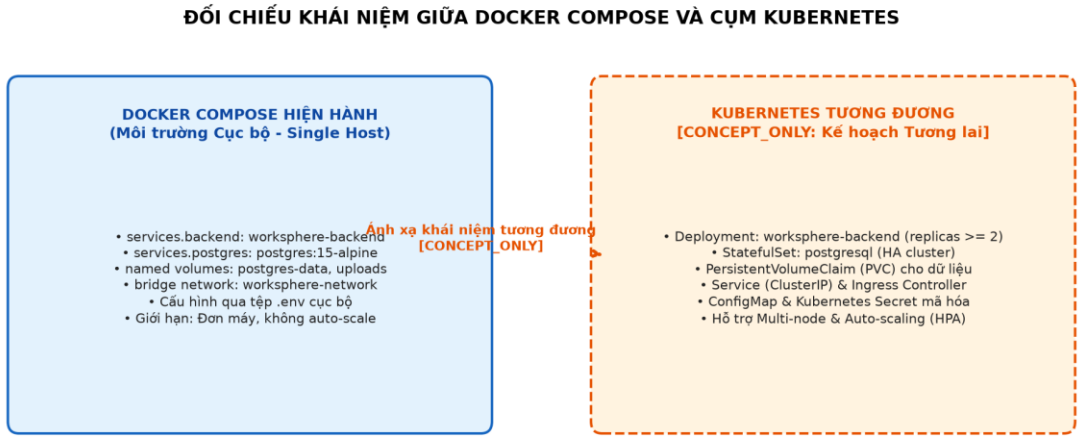
[CONCEPT_ONLY] Phân tích lý thuyết ứng dụng vào định hướng phát triển của WorkSphere [CONCEPT_ONLY]:

- • Mô hình IaaS (Infrastructure as Service - ví dụ AWS EC2, GCP Compute Engine): Cung cấp máy ảo, người quản trị phải tự cài đặt Docker runtime, cấu hình tường lửa và giám sát.
- • Mô hình PaaS (Platform as Service - ví dụ AWS ECS, Google Cloud Run): Cho phép đưa trực tiếp Docker image lên chạy, nhà cung cấp đám mây tự động quản lý hạ tầng máy chủ bên dưới và hỗ trợ tính năng tự động co giãn (Auto-scaling). Đây là mô hình được nhóm khuyến nghị cho giai đoạn Staging tương lai.

10.2. Đối Chiếu Khái Niệm Giữa Docker Compose Và Cụm Kubernetes

[CONCEPTUAL_MAPPING] Bảng ánh xạ khái niệm kỹ thuật giữa môi trường Compose hiện tại và cụm Kubernetes tương lai [CONCEPTUAL_MAPPING]:

Khái niệm trong Docker Compose	Đối tượng tương đương trong Kubernetes	Ý nghĩa kiến trúc & Sự khác biệt
services.backend	Deployment (replicas >= 2)	K8s quản lý nhiều Pods cùng lúc, tự phục hồi khi Pod chết
services.postgres	StatefulSet	K8s đảm bảo thứ tự định danh và ổ đĩa gắn kết cố định cho CSDL
named volumes (postgres-data)	PersistentVolumeClaim (PVC)	K8s trừu tượng hóa ổ lưu trữ đĩa mạng độc lập với Node tính toán
bridge network (worksphere-net)	Service (ClusterIP)	K8s cung cấp Virtual IP nội bộ và cân bằng tải giữa các Pods
ports: '8080:8080'	Ingress Controller	K8s định tuyến HTTP/HTTPS domain bên ngoài vào các Service
environment file (.env)	ConfigMap & Kubernetes Secret	K8s phân tách dữ liệu cấu hình thường và khóa bí mật mã hóa



Hình 10.1: Sơ đồ đối chiếu khái niệm giữa Docker Compose và Kubernetes

Khẳng định ranh giới: Dự án WorkSphere hiện tại HOÀN TOÀN CHƯA TRIỂN KHAI Kubernetes manifests trong mã nguồn; bảng đối chiếu trên phục vụ nghiên cứu lý thuyết theo yêu cầu học phần.

CHƯƠNG 11: AN TOÀN CHUỖI CUNG ỨNG PHẦN MỀM (SUPPLY-CHAIN SECURITY)

11.1. Phòng Chống Tấn Công Chuỗi Cung Ứng Mã Nguồn

Bảo vệ tính toàn vẹn của mã nguồn từ gốc:

- • Khóa phiên bản phụ thuộc (Lockfiles): Sử dụng tệp `package-lock.json` trên Mobile và Maven dependency hash để ngăn chặn nguy cơ cài đặt phải các gói mã độc mạo danh (Typosquatting).
- • Quy trình cô lập tài nguyên bên ngoài (.external/quarantine): Mọi tài nguyên, template hoặc công cụ từ bên ngoài đưa vào dự án đều phải lưu tại thư mục cách ly, quét chữ ký số SHA-256 và rà soát an ninh trước khi sử dụng.

11.2. Phân Định Chữ Ký Số Phát Hành (Debug vs Release Signing)

Dự án phân biệt rõ ràng giữa hai cơ chế ký nhị phân Android:

- • Bản Debug APK: Ký bằng khóa phát triển mặc định của Android SDK (`debug.keystore`), chỉ phục vụ kiểm thử nội bộ trên máy ảo Emulator hoặc máy thử nghiệm của nhóm.
- • Bản Release AAB/APK thương mại: Đòi hỏi khóa ký phát hành bảo mật cao (Release Keystore), yêu cầu lưu trữ bí mật trong phần cứng hoặc dịch vụ quản lý khóa (KMS). Dự án hiện tại CHƯA KÝ BẢN RELEASE THƯƠNG MẠI.

CHƯƠNG 12: ĐÁNH GIÁ KẾT QUẢ, HẠN CHẾ VÀ LỘ TRÌNH PHÁT TRIỂN

12.1. Đối Chiếu Kết Quả Triển Khai Thực Tế

Hệ thống triển khai phần mềm của WorkSphere đã hoàn thành xuất sắc các mục tiêu cốt lõi:

- • Tự động hóa hoàn toàn chuỗi CI trên GitHub Actions với 5 jobs, phát hiện lỗi lập trình ngay lập tức.
- • Đóng gói Docker image tối ưu dung lượng (~280MB) và đảm bảo an toàn thực thi phi đặc quyền (UID 10001).
- • Vận hành ổn định trên Docker Compose với cơ chế tự phục hồi, giám sát Actuator/Prometheus và quy trình sao lưu toàn vẹn.

12.2. Các Hạn Chế Hiện Hành [CURRENT LIMITATIONS]

[CURRENT_LIMITATIONS] Nhóm ghi nhận trung thực các hạn chế kỹ thuật hiện tại:

- **1.** Chưa có môi trường Staging/Production trên điện toán đám mây công cộng.
- **2.** Chưa xuất bản Docker image lên các Container Registry chính thức (Docker Hub, GitHub Container Registry).
- **3.** Chưa triển khai giao diện trực quan hóa Grafana và hệ thống gửi cảnh báo tự động (Alertmanager/Slack).
- **4.** Chỉ số RPO và RTO mới dừng ở mức xác thực diễn tập cục bộ, chưa ban hành chỉ số cam kết vận hành doanh nghiệp chính thức.

12.3. Lộ Trình Phát Triển Môi Trường Staging & Production

- • Giai đoạn tiếp theo (Staging Readiness): Đăng ký tài khoản hạ tầng đám mây (AWS/GCP), xuất bản Docker image có gắn nhãn phiên bản Semantic Versioning (`v1.0.0`), cấu hình chứng chỉ bảo mật HTTPS/TLS.
- • Giai đoạn nâng cao (Production Operability): Xây dựng cụm Kubernetes (EKS/GKE), tích hợp hệ thống cảnh báo qua Slack/Telegram, thiết lập quy trình sao lưu tự động hàng ngày lên đám mây lưu trữ S3.

TÀI LIỆU THAM KHẢO (REFERENCES)

- **1.** Trường Đại học CMC, Khoa Công nghệ Thông tin & Truyền thông, Bộ bài giảng 9 tuần học phần Triển khai phần mềm (DevOps), Hà Nội, 2026. [LOCAL_ONLY_REFERENCE, KHÔNG PHÂN PHỐI RA NGOÀI REPOSITORY]
- **2.** Docker Inc., Docker Documentation & Multi-Stage Builds Guide, https://docs.docker.com, 2026.
- **3.** GitHub Inc., GitHub Actions Workflow Syntax & Continuous Integration Guide, https://docs.github.com/actions, 2026.
- **4.** Spring Boot Team, Spring Boot Actuator: Production-ready Features, VMware Tanzu, 2026.
- **5.** Prometheus Authors, Prometheus Monitoring System & Time Series Database, The Linux Foundation, 2026.
- **6.** Kubernetes Authors, Kubernetes Documentation & Core Concepts, Cloud Native Computing Foundation (CNCF), 2026.
- **7.** WorkSphere Engineering Team, Tài liệu Hướng dẫn Chạy và Vận hành Hệ thống WorkSphere, reports/WORKSPHERE_PROJECT_OVERVIEW_AND_RUN_GUIDE.md, 2026.

PHẦN PHỤ LỤC: PHÂN CÔNG BIÊN SOẠN & THUYẾT TRÌNH ĐỀ XUẤT

[PROPOSED_ASSIGNMENT_PENDING_USER_APPROVAL] *(Trạng thái phân công: PROPOSED_ASSIGNMENT_PENDING_USER_APPROVAL — Phân công biên soạn và thuyết trình đề xuất chờ Người dùng phê duyệt)*:

Họ và Tên	Mã SV	Nội dung biên soạn chuyên môn đề xuất	Phần thuyết trình dự kiến
Nguyễn Minh Hiếu	BIT247729	Tổng quan DevOps, Chuỗi CI/CD tích hợp, Báo cáo và Đánh giá kết luận	Chương 1, 3, 4, 12 & Trả lời vấn đáp
Nguyễn Văn Cường	BIT240048	Quản lý mã nguồn SCM, Git branching, Cấu hình môi trường, Docker Compose	Chương 2, 6 & Demo Docker Compose
Vũ Thị Thu Hướng	BIT240113	Quy trình sao lưu (Backup), Diễn tập phục hồi (Restore), Runbooks vận hành	Chương 9 & Demo khôi phục dữ liệu
Hoàng Bảo Long	BIT240142	An toàn pipeline CI, Giám sát Actuator/Prometheus, Request Correlation MDC	Chương 8, 11 & Demo Prometheus
Nguyễn Đức Khiêm	BIT240125	Đóng gói Docker Multi-stage, Non-root, Rollback và Phân tích Kubernetes/Cloud	Chương 5, 7, 10 & Demo Dockerfile